Can a green Python release pipeline be trusted?

Soup DevOps internship - Amir - 12 November 2026

Verdict and acceptance status

Automate TestPyPI after the gates pass; keep human approval for real PyPI. The implementation and local incident evidence are complete. Hosted acceptance is still pending: GitHub returned internal startup failures with zero jobs, and TestPyPI publisher registration has not been confirmed. A red startup result is not evidence that a package bug was caught. No production package or deliberately broken fixture was published.

Repository: soup-devops-release-lab. The complete evidence and every retained remote attempt are in the evidence ledger.

Four incidents, one weak pipeline

The CLI reads a TOML service config and bundled JSON defaults, validates it, and prints JSON. Fixed version: 0.9.4; isolated faulty fixtures: 0.9.3. Naive CI uses a Linux Python 3.12 editable install, two library tests, and a build. All four fixtures passed those checks and built successfully in the recorded local runs.

Fault	What users see	Hardened consumer that rejects it
Python compatibility	Unconditional tomllib import fails on 3.10	Python 3.10 CLI launch
OS compatibility	Unix-only pwd import prevents Windows startup	Native Windows CLI launch
Missing wheel data	Source has defaults.json; installed wheel does not	inspect against an installed wheel
Broken entry point	Console command targets missing_main	Invoke the installed executable

Each rejection log contains the intended error signature. The fixed wheel and sdist passed six clean consumers: Linux 3.10, Linux 3.12, and Windows 3.13, two formats each. Local quality checks passed 38 tests with 96.15% combined line/branch coverage. The unprivileged Docker runtime launched with network disabled, read-only filesystem, and capabilities dropped.

What changed in the release path

Actions lint/type checks and unit tests accompany one build. The fixed matrix covers Python 3.10-3.14 on Linux, Windows, and macOS; this full hosted matrix has not yet executed successfully. Consumers use fresh venvs outside the checkout, install the wheel or rebuild the sdist, verify the import location, launch both entry points, and test valid/invalid configs including a Unicode path. Both distribution identity and SHA-256 hashes are checked. A version tag must equal built metadata and installed CLI output.

Only the publisher gets OIDC permission, after every check. It downloads the exact same-run artifact ID, verifies commit/tag/hashes, and uploads those bytes through the SHA-pinned PyPA action. It does no source checkout or build. A separate read-back downloads TestPyPI's public distributions, rejects extra files or changed bytes, and tests them again. Read-back can detect a bad publication; it cannot reverse one.

Silent failures and the remaining trust boundary

Implemented controls close at least three concrete paths: exact transitive hash locks plus one explicit index reduce dependency swaps; fresh installs without restored caches expose resolver errors; immutable artifact IDs and commit/hash checks reject stale bundles; fork PR checks have read-only permissions and no secrets/OIDC; exact tag/metadata matching prevents version drift. Actions and the Docker base are pinned. These were prioritized because source tests do not exercise identity or credential boundaries.

Hashes preserve chosen bytes, including malicious ones. Initial dependency selection, unknown vulnerabilities, compromised maintainers/runners, untested architectures, system libraries, user environments, and behavior outside these fixtures remain risks. Attestations prove origin, not correctness. Downstream pip users do not automatically inherit our runtime hash lock. Repository branch/tag protections and independent review remain prerequisites for real PyPI.

GPU release and nightly design

This CLI needs no GPU; the requested design covers a future CUDA component and has not rented hardware. Use ephemeral Runpod Secure Cloud RTX 4090 instances at the checked $0.89/hour rate. Initially budget one validated software tuple: two monthly releases at 20 test minutes plus 3 startup minutes, and 30 nightlies at 10 plus 3. Total: 436 minutes = $6.47; add 20% retry reserve and $1 small storage/log allowance: $8.76/month. Existing broker/watchdog hosting is assumed. Larger compatibility sets require rebudgeting.

Allocate only after CPU gates. An OIDC broker admits protected refs, enforces one pod and a $15 GPU allocation ceiling, and never gives provider credentials to the runner. Separate release/nightly queues protect pending releases. Use 25/15-minute TTLs, an independent minute-by-minute termination watchdog with heartbeat alerts, signed result upload URLs, and reconciliation after ambiguous API responses. Cancel superseded nightlies; retry only transient infrastructure failures once. Slow startup, provider outages and retained storage can defeat a naive cost estimate; the ledger is not a provider hard cap.

What to fix first

First obtain real hosted red/green and OIDC publication/read-back evidence. Next protect source, workflow, lock, and tag changes through independent review and account recovery. Then implement a durable release circuit breaker and broaden compatibility coverage. Today a maintainer must manually disable later releases after a publish/read-back failure. Keep the production approval gate until those controls are demonstrated.

Reflection and AI use

The observed surprise was that editable tests hid failures in installed artifacts, while familiar pinned tool versions also failed a current advisory audit. Those initial pins were replaced and the failed audit retained. The main concern remains that one repository controls both artifact production and its evidence. Codex wrote and tested this implementation; OpenCode and Antigravity were requested as reviewers, with their actual outcomes recorded. The applicant still needs to review the work personally and adapt the reflection honestly. Live-discussion answers were not prewritten.

Sources: PyPI OIDC, GitHub security, Runpod pricing. Detailed threat model, GPU assumptions, disclosure, and reproduction commands are in the repository.